

Measuring an Asynchronous Report API

An empirical comparison of acknowledgement time and completion time

Duration	Work mode	Code
75 minutes	Pairs	Provided

Central question: Does the asynchronous contract keep acknowledgement time independent of report-generation duration?

Learning outcomes

- Measure acknowledgement time separately from completion time.
- Trace a submitted request through queued, processing and terminal job states.
- Compare asynchronous measurements with the synchronous results from Lab 1.
- Evaluate whether the new contract satisfies its stated requirements.
- Avoid the incorrect conclusion that asynchronous execution makes work faster.

The revised contract

POST /v1/reports no longer promises the completed report. It promises to acknowledge accepted work and return a stable job identity.

POST promise 202 Accepted means the report request was accepted for processing. It does not mean that report generation completed or succeeded.

The client later uses GET /v1/jobs/{id} to observe queued, processing, completed or failed and to retrieve the result when available.

Experimental design

Research question

How does artificial report-generation delay affect (1) POST acknowledgement time and (2) total job-completion time in the asynchronous API?

Hypothesis — complete before testing

If report-generation delay increases, then acknowledgement time will remain the same, while completion time will increase, because the acknowledgement time is decoupled from the report generation, while completion is dependent on the report-generation time.

Variables

Role	Variable
Independent	Artificial worker delay: 0 s, 3 s, 7 s, 12 s
Dependent	POST acknowledgement time, total completion time, poll count, observed states and final state
Controlled	Endpoint, payload, consumer, database, worker count, polling interval, client script and server configuration

Two clocks

Measure	Start	Stop	What it tests
T_ack	Immediately before POST	202 response received	Whether submission waits for report work
T_complete	Immediately before POST	completed or failed observed	How long the entire operation takes

Key distinction: The new contract should reduce or stabilize T_ack. It does not necessarily reduce T_complete.

Before you measure: predict

Record predictions before running any condition. Use the synchronous evidence from Lab 1, but do not copy its conclusion automatically—the contract has changed.

Delay	Predicted T _{ack}	Predicted T _{complete}	Expected final state
0 s	>1s	≈1s	200 OK
3 s	>1s	≈4s	200 OK
7 s	>1s	≈7s	200 OK
12 s	>1s	≈12s	200 OK

Contract reconstruction

What fact does 202 Accepted communicate?

202 Accepted communicates the fact that the request was received but the work hasn't been completed as yet. It is just an acknowledgement.

What fact does it not communicate?

It does not communicate the fact that the work succeeds or will succeed.

Why must the server return a job identifier?

To verify that the request has been received but the request also ends when the work is done so it's the only way a client can find the identifier of that piece of work after the fact. It is essentially a receipt or proof of purchase that is used to find it after processing.

Which request retrieves the eventual outcome?

GET /v1/jobs/{id} is the request that eventually receives the outcome.

Which component performs the report work?

cmpd/api/worker.go performs the report work as opposed to last lab having .../[reports.go](#) perform the report work

Procedure

Part A — Verify the supplied system

- The POST handler creates a queued job and immediately returns 202 Accepted.
- The worker applies the configured artificial delay, generates the report and records the result.
- GET /v1/jobs/{id} returns the job's current state.
- The measurement script polls every 250 ms until completed or failed.

Controlled condition The `-report-delay` value must be applied by the worker—not by the POST handler. Delaying the POST handler would recreate the synchronous waiting problem.

Part B — Create request.json

```
{
  "consumer_id": "0198f000-0000-7000-8000-000000000001",
  "from": "2026-01-01T00:00:00Z",
  "to": "2027-01-01T00:00:00Z"
}
```

Part C — Start one condition

```
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=0s
# Repeat later with: 3s, 7s and 12s
```

Measurement script

Save the supplied script as `measure_async.sh`. Read it before running it: identify where `T_ack` starts and stops, where `T_complete` stops, and how terminal state is detected.

```
#!/usr/bin/env bash
set -u

api='http://localhost:4000/v1'
start_ns=$(date +%s%N)

submit=$(curl --silent --show-error \
  --write-out '%\n{time_total}' \
  --request POST \
  --header 'Content-Type: application/json' \
  --data @request.json \
  "$api/reports")

ack_time=${submit##*'\n'}
body=${submit%*\n*}
job_id=$(printf '%s' "$body" | jq -r '.job_id')
polls=0

while true; do
  job=$(curl --silent --show-error "$api/jobs/$job_id")
  status=$(printf '%s' "$job" | jq -r '.job.status')
  polls=$((polls + 1))
  printf 'poll=%d status=%s\n' "$polls" "$status"
  case "$status" in completed|failed) break ;; esac
  sleep 0.25
done

end_ns=$(date +%s%N)
complete_time=$(awk -v s="$start_ns" -v e="$end_ns" \
  'BEGIN { printf "%.3f", (e-s)/1000000000 }')
printf 'job_id=%s\nack_time=%ss\ncomplete_time=%ss\npolls=%d\nfinal=%s\n' \
  "$job_id" "$ack_time" "$complete_time" "$polls" "$status"
```

Make it executable and run it:

```
chmod +x measure_async.sh
./measure_async.sh
```

Run the controlled experiment

1. Run the script three times for the current delay.
2. Record T_{ack} , $T_{complete}$, poll count and final state.
3. Inspect at least one GET response in each condition.
4. Restart the server with the next delay.
5. Do not run different delay conditions simultaneously.

Condition: 0 seconds

Run	T_{ack} (s)	$T_{complete}$ (s)	Polls	Final state	States observed
1	0.002465s	0.025s	1	completed	Started -> completed
2	0.002588s	0.283s	2	completed	Started -> completed
3	0.002165s	0.282s	2	completed	Started -> completed

```
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=0s
time=2026-08-28T14:07:26.521-06:00 level=INFO msg="database connection pool established"
time=2026-08-28T14:07:26.521-06:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-08-28T14:18:52.775-06:00 level=INFO msg="report job started" job_id=9b699562-ee6b-44d0-bc94-ea9e0697fcf9 artificial_delay=0s
time=2026-08-28T14:18:52.779-06:00 level=INFO msg="report job completed" job_id=9b699562-ee6b-44d0-bc94-ea9e0697fcf9
time=2026-08-28T14:35:35.024-06:00 level=INFO msg="report job started" job_id=24df5559-6348-445b-b021-21c587d8e740 artificial_delay=0s
time=2026-08-28T14:35:35.026-06:00 level=INFO msg="report job completed" job_id=24df5559-6348-445b-b021-21c587d8e740
time=2026-08-28T14:35:37.274-06:00 level=INFO msg="report job started" job_id=f36e981b-a3b9-41a9-b426-c1feea6c529a artificial_delay=0s
time=2026-08-28T14:35:37.276-06:00 level=INFO msg="report job completed" job_id=f36e981b-a3b9-41a9-b426-c1feea6c529a
time=2026-08-28T14:35:38.274-06:00 level=INFO msg="report job started" job_id=6fe2c4b2-ed1a-4126-ac73-5459f3326431 artificial_delay=0s
time=2026-08-28T14:35:38.276-06:00 level=INFO msg="report job completed" job_id=6fe2c4b2-ed1a-4126-ac73-5459f3326431
```

```
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ ./measure_async.sh
poll=1 status=completed
job_id=24df5559-6348-445b-b021-21c587d8e740
ack_time=0.002465s
complete_time=0.025s
polls=1
final=completed
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ ./measure_async.sh
poll=1 status=queued
poll=2 status=completed
job_id=f36e981b-a3b9-41a9-b426-c1feea6c529a
ack_time=0.002588s
complete_time=0.283s
polls=2
final=completed
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ ./measure_async.sh
poll=1 status=queued
poll=2 status=completed
job_id=6fe2c4b2-ed1a-4126-ac73-5459f3326431
ack_time=0.002165s
complete_time=0.282s
polls=2
final=completed
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ |
```

Condition: 3 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final state	States observed
1	0.002597s	3.412s	14	completed	Started -> Completed
2	0.002031s	3.148s	13	completed	Started -> Completed
3	0.002144s	3.411s	14	completed	Started -> Completed

```
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=3s
time=2026-08-28T14:40:08.918-06:00 level=INFO msg="database connection pool established"
time=2026-08-28T14:40:08.919-06:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-08-28T14:40:15.671-06:00 level=INFO msg="report job started" job_id=e903cee4-fcc4-4a20-934a-701a1f8f42ba artificial_delay=3s
time=2026-08-28T14:40:18.674-06:00 level=INFO msg="report job completed" job_id=e903cee4-fcc4-4a20-934a-701a1f8f42ba
time=2026-08-28T14:40:21.171-06:00 level=INFO msg="report job started" job_id=258ee3b4-986a-4fea-86f8-cd5d0a748077 artificial_delay=3s
time=2026-08-28T14:40:24.173-06:00 level=INFO msg="report job completed" job_id=258ee3b4-986a-4fea-86f8-cd5d0a748077
time=2026-08-28T14:40:25.672-06:00 level=INFO msg="report job started" job_id=28cfc15e-4280-4b1c-ab73-f04564e7cb15 artificial_delay=3s
time=2026-08-28T14:40:28.674-06:00 level=INFO msg="report job completed" job_id=28cfc15e-4280-4b1c-ab73-f04564e7cb15
```

```
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ ./measure_async.sh
poll=1 status=queued
poll=2 status=processing
poll=3 status=processing
poll=4 status=processing
poll=5 status=processing
poll=6 status=processing
poll=7 status=processing
poll=8 status=processing
poll=9 status=processing
poll=10 status=processing
poll=11 status=processing
poll=12 status=processing
poll=13 status=processing
poll=14 status=completed
job_id=e903cee4-fcc4-4a20-934a-701a1f8f42ba
ack_time=0.002597s
complete_time=3.412s
polls=14
final=completed
```

```
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ ./measure_async.sh
poll=1 status=queued
poll=2 status=processing
poll=3 status=processing
poll=4 status=processing
poll=5 status=processing
poll=6 status=processing
poll=7 status=processing
poll=8 status=processing
poll=9 status=processing
poll=10 status=processing
poll=11 status=processing
poll=12 status=processing
poll=13 status=completed
job_id=258ee3b4-986a-4fea-86f8-cd5d0a748077
ack_time=0.002031s
complete_time=3.148s
polls=13
final=completed
```

```
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ ./measure_async.sh
poll=1 status=queued
poll=2 status=processing
poll=3 status=processing
poll=4 status=processing
poll=5 status=processing
poll=6 status=processing
poll=7 status=processing
poll=8 status=processing
poll=9 status=processing
poll=10 status=processing
poll=11 status=processing
poll=12 status=processing
poll=13 status=processing
poll=14 status=completed
job_id=28cfc15e-4280-4b1c-ab73-f04564e7cb15
ack_time=0.002144s
complete_time=3.411s
polls=14
final=completed
```

Condition: 7 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final state	States observed
1	0.002617s	7.324s	29	Completed	Started -> Completed
2	0.002251s	7.322s	29	Completed	Started -> Completed
3	0.002295s	7.324s	29	Completed	Started -> Completed

```
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=7s
time=2026-08-28T14:46:03.962-06:00 level=INFO msg="database connection pool established"
time=2026-08-28T14:46:03.962-06:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-08-28T14:46:15.214-06:00 level=INFO msg="report job started" job_id=2927dec6-62df-4cbb-9283-290db8f2edb5 artificial_delay=7s
time=2026-08-28T14:46:22.218-06:00 level=INFO msg="report job completed" job_id=2927dec6-62df-4cbb-9283-290db8f2edb5
time=2026-08-28T14:46:24.965-06:00 level=INFO msg="report job started" job_id=3f4092ba-1361-4e97-a651-307dbb4b44d5 artificial_delay=7s
time=2026-08-28T14:46:31.967-06:00 level=INFO msg="report job completed" job_id=3f4092ba-1361-4e97-a651-307dbb4b44d5
time=2026-08-28T14:46:36.214-06:00 level=INFO msg="report job started" job_id=b67e467e-24dc-41b7-ae37-157b6feb8c54 artificial_delay=7s
time=2026-08-28T14:46:43.217-06:00 level=INFO msg="report job completed" job_id=b67e467e-24dc-41b7-ae37-157b6feb8c54
```

Condition: 12 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final state	States observed
1	0.003099s	12.279s	48	Completed	Started -> Completed
2	0.002264s	12.278s	48	Completed	Started -> Completed
3	0.002192s	12.282s	48	Completed	Started -> Completed

```
rsam@DESKTOP-98VU05G:~/projects/advweb/lab/gatekeeper-asynchronous-code$ go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=12s
time=2026-08-28T14:51:14.234-06:00 level=INFO msg="database connection pool established"
time=2026-08-28T14:51:14.234-06:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-08-28T14:51:27.237-06:00 level=INFO msg="report job started" job_id=925adfa5-3df8-4746-af75-06068904c01d artificial_delay=12s
time=2026-08-28T14:51:39.241-06:00 level=INFO msg="report job completed" job_id=925adfa5-3df8-4746-af75-06068904c01d
time=2026-08-28T14:51:41.737-06:00 level=INFO msg="report job started" job_id=4966349a-2dab-4351-ab4c-f6b16af91cbd artificial_delay=12s
time=2026-08-28T14:51:53.740-06:00 level=INFO msg="report job completed" job_id=4966349a-2dab-4351-ab4c-f6b16af91cbd
time=2026-08-28T14:51:56.487-06:00 level=INFO msg="report job started" job_id=0bf6ff57-976f-4086-9466-526608b20aef artificial_delay=12s
time=2026-08-28T14:52:08.490-06:00 level=INFO msg="report job completed" job_id=0bf6ff57-976f-4086-9466-526608b20aef
```

Summarize and compare

Calculate the mean for each clock: $\text{mean} = (T_1 + T_2 + T_3) \div 3$.

Delay	Mean T_ack	Mean T_complete	Successful jobs
0 s	≈ 0.002406s	≈ 0.197s	3 / 3
3 s	≈ 0.002257s	≈ 3.324s	3 / 3
7 s	≈ 0.002388s	≈ 7.323s	3 / 3
12 s	≈ 0.002518s	≈ 12.280s	3 / 3

Compare with Lab 1

Delay	Synchronous response time	Async T_ack	Async T_complete
0 s	≈ 0.003588s	≈ 0.002406s	≈ 0.197s
3 s	≈ 2.900447s	≈ 0.002257s	≈ 3.324s
7 s	≈ 6.695139s	≈ 0.002388s	≈ 7.323s
12 s	≈ 11.499333s	≈ 0.002518s	≈ 12.280s

Observation

Write what changed numerically without explaining the mechanism.

Async T_ack remained consistent throughout measurements as it's decoupled from any work done and is sent as soon as the request reaches the worker, however the total overall completion time remained largely unchanged with the exception of Async T_complete taking slightly longer than all instances of Synchronous response time with the same delay.

Explanation

Why did T_ack behave differently from synchronous response time?

In the synchronous report generation, response time is included with the work done for the full generation report time with the handler being unable to response until the work was done, while in the asynchronous report generation, T_ack only measures the time the handler takes to queue the job and responds as soon as it's done, regardless of the status of the actual work being done.

Why did T_complete still change with artificial delay?

T_complete measures the time until the job reaches an end state (being completed or incomplete/dropped). The worker cannot mark the end state of a job/request until the artificial delay is finished, which is dependent on the length of the artificial delay.

Which component now owns the long-running work?

The component that now owns the long-running work is `worker.go` instead of `report.go` in `cmd/api/`. `reports.go` is now simply the POST handler that sends back the resource to the client instead of doing the work on that resource.

What evidence shows that the original request ended before the report completed?

In the server logs, it shows statuses for each report with "report job started" and "report job completed" with a time gap longer than the time for T_ack in each artificial delay. This shows that request was sent and closed before the job was done, and the response to that request was sent after that time.

Evaluate the contract

For each requirement, use your measurements—not intuition—to decide whether the async design provided the property.

Requirement	Evidence	Supported?
Acknowledge without waiting for report completion	T_ack stayed consistent throughout all artificial time delay parameters	Yes
Allow work to continue after POST ends	Server logs show job processing starting/completing after the POST response	Yes
Expose current job state	In the measure_async.sh, the responses showed queued, processing and completed states across all polls	Yes
Expose completed result or failure	All 12 jobs completed and returned that result, no job showed any signs of failure so no real way to know if it can expose a result failure.	Unclear

Limitations and new questions

State two properties this experiment did not test—for example worker-crash recovery, duplicate submissions, authorization, retention or multiple-worker contention.

- 1) We never tested a failed job state to see if we upheld the contract's ability to expose failure outcomes.
- 2) Only one job was submitted at a time and sequentially. We did not test worker behavior under concurrent submissions or multiple simultaneous jobs to test the contract's ability to lock jobs.

In-class exit ticket

Element	Your response
Claim	Did the async contract decouple acknowledgement from report duration? Yes. The async contract successfully decoupled acknowledgment from report duration
Evidence	Provide one T_ack comparison and one T_complete comparison. T_ack remained consistent across all time delays, while T_complete increased proportionally to the artificial time delay.
Reasoning	Explain the mechanism that produced the pattern. The artificial delay was moved from synchronous' report.go routine and placed into worker.go work routine, which decouples the response from the work but still affects the actual work done by worker.go which has to send the end state back to report.go . This allows report.go to update on the state throughout the entire process.
Limitation	Name one conclusion the experiment cannot support.

Element	Your response
	The experiment cannot support the conclusions on exposing job failure states or locking behavior for concurrent job submissions.

Complete outside class

Submit a short comparative laboratory report containing:

- Research question and preregistered hypothesis
- Variables, controls and polling interval
- Raw asynchronous measurements and calculated means
- A comparison with the synchronous results from Lab 1
- Observation separated from causal explanation
- A requirement-by-requirement contract evaluation
- At least two experimental limitations
- One next experiment that would test an unresolved property

Required reasoning chain

Layer	Question
Evidence	What did <code>T_ack</code> and <code>T_complete</code> do as work duration increased?
Requirement	Which required property was tested?
Contract	What did POST promise, and what did GET promise?
Implementation	Which mechanism produced the observed behavior?
Evaluation	Did the evidence support the contract claim?

Final distinction: Asynchronous execution can make the client free sooner without making the report finish sooner. The experiment must report both facts.